



INSTITUTO TECNOLÓGICO SUPERIOR DE LERDO

Nombre del alumno:

Alexander Adonai Molina Rodríguez - 242310131

Nombre del profesor:

Jesús Salas Marín

Nombre de la materia:

Administración y Organización de Datos

Carrera:

Ing. Informática

Fecha:

03/06/2026

Turno:

Matutino

SISTEMA DE GESTIÓN DE ÓRDENES (RESTAURANT PROJECT)

1. Portada e Introducción Ejecutiva

Este informe técnico presenta los fundamentos de diseño y la evaluación del sistema web de gestión de órdenes desarrollado para el entorno de un restaurante de comida mexicana. El ecosistema del proyecto está integrado por un script en Python (generar.py) encargado de la persistencia de datos iniciales mediante la simulación de transacciones reales, una interfaz de usuario estructurada en HTML5 y CSS puro para la captura y visualización de datos de manera intuitiva, y un backend basado en PHP para el control de flujos, validación en servidor, parseo y generación de archivos temporales de consulta.

El desarrollo e implementación de esta arquitectura demuestra de forma directa la adquisición de la competencia del semestre: "Evaluar diferentes organizaciones de archivos aplicándolas a situaciones reales". Al enfrentarnos al problema operativo de un restaurante, se descartó el uso tradicional de un Sistema Gestor de Bases de Datos Relacionales (RDBMS) para analizar el comportamiento físico de los datos en disco. El proyecto demuestra la capacidad de diseñar estructuras de archivos indexadas implícitamente por posición, segmentar registros mediante delimitadores específicos, aplicar filtros en memoria secundaria sin saturar el servidor y evaluar el impacto computacional en términos de velocidad de lectura/escritura y almacenamiento de datos, validando que una solución basada en archivos planos es viable y óptima para escenarios de negocio controlados.

2. Justificación de Formatos

Archivos Planos Delimitados (.txt): Se seleccionó este formato para el almacenamiento de maestro.txt y filtrado.txt utilizando la tubería (|) como delimitador. La elección responde a que los archivos secuenciales planos optimizan los tiempos de escritura cíclica al operar en modo de adición directa (append) o sobreescritura limpia, evitando la sobrecarga (overhead) estructural de los formatos relacionales. Al dividir las cadenas con la función explode(), se logra un acceso directo en memoria a cada campo de la orden, reduciendo la complejidad del código.

Hojas de Estilo en Cascada (.css): El formato CSS puro se eligió por encima de frameworks externos para garantizar que la carga visual no dependa de peticiones HTTP a servidores de terceros ni de la descarga de librerías densas. Esto mantiene el tiempo de respuesta del renderizado del navegador al mínimo y asegura el control absoluto sobre la visualización corporativa del restaurante.

3. Estimación de Uso y Escalabilidad (Proyección Numérica)

El script de simulación demostró que la creación de 300 registros base genera un archivo plano sumamente ligero (menos de 20 KB de almacenamiento en disco). El procesamiento de estos 300 registros se realiza de forma casi instantánea mediante el uso de la función lineal de lectura por líneas.

¿Qué pasa si el sistema escala y recibe una cantidad grande de registros N?

- **Complejidad Temporal:** Al utilizar una organización secuencial pura donde PHP debe leer línea por línea mediante `fgets()` o cargar todo en un arreglo con `file()`, el tiempo de búsqueda crece de manera directamente proporcional al número de registros.
- **Proyección:** Si el restaurante registra un promedio de 10,000 órdenes al mes, el archivo pesará aproximadamente 650 KB. A este nivel, el procesamiento en el servidor seguirá tardando milisegundos. Sin embargo, si el archivo acumula 500,000 registros, el consumo de memoria RAM del servidor web aumentará notablemente al procesar los filtros y la función `explode()`, provocando un ligero retraso perceptible en la visualización de la tabla HTML dinámica. Para mitigar este comportamiento en un entorno de producción masivo, se requeriría implementar un archivo indexado que permita búsquedas directas en tiempo constante u omitir la carga completa del archivo en memoria principal.

4. Análisis Costo-Beneficio

Criterio de Evaluación	Costo (Técnico / Computacional)	Beneficio Operativo
Infraestructura y Servidor	Bajo: No se requiere pagar licencias de software comerciales ni mantener bases de datos activas las 24 horas. El sistema corre en cualquier servidor web básico con soporte PHP.	Alto: Reducción de costos de mantenimiento técnico. Flexibilidad para mover el sistema de servidor de manera inmediata mediante copia de archivos.
Complejidad de Almacenamiento	Moderado: El uso de archivos planos incrementa ligeramente el tamaño de almacenamiento en texto en comparación con	Alto: Trazabilidad absoluta. Los datos son legibles por cualquier auditor y permiten descargas locales inmediatas para

	formatos binarios comprimidos.	respaldos externos en formato accesible.
Tiempo de Procesamiento	Bajo/Medio: El proceso de búsqueda lineal consume procesamiento de CPU al parsear cadenas de texto mediante funciones de software nativas.	Muy Alto: El tiempo de respuesta para el usuario final se reduce drásticamente en comparación con una búsqueda manual en papel. Se pasa de buscar una orden en minutos a localizarla en una fracción de segundo.
Desarrollo y Mantenimiento	Bajo: Arquitectura modular y limpia sin dependencias externas complejas, lo que reduce las líneas de código propensas a errores.	Alto: El personal del restaurante cuenta con una herramienta centralizada que disminuye los errores humanos de captura y agiliza el flujo de atención en comedores.

5. Conclusiones de Ingeniería

El desarrollo de este sistema permite comprender los fundamentos de bajo nivel del manejo de la información en entornos informáticos. Antes de transicionar hacia el uso de los Sistemas Gestores de Bases de Datos Relacionales (materia que corresponde al siguiente semestre académico), es de vital importancia comprender cómo el sistema operativo manipula los archivos físicos a través de operaciones primitivas de apertura, lectura, escritura y cierre (fopen, fgets, fwrite, fclose).

Se concluye que las organizaciones de archivos secuenciales y planos representan una excelente alternativa de ingeniería cuando los requerimientos del problema exigen ligereza, portabilidad y rapidez de implementación. La realización de este proyecto consolida el criterio técnico necesario para evaluar cuándo una solución simple basada en archivos de texto es suficiente para cubrir las necesidades reales de una organización, y cuándo la escala del problema justifica la inversión en infraestructuras de bases de datos de mayor complejidad.